SRS — v1.16 (05/16/2026)

1. Introduction

This document defines a simplified Software Requirements Specification for the TaskAnchor system. It focuses on a minimal, achievable MVP that supports task follow-through, visibility, and continuity of work.

The system ensures that tasks are not only captured, but consistently revisited and completed over time.

MVP feature development is complete. Current remaining capstone work is limited to validation, documentation alignment, Week 8 user feedback testing, Week 9 bug fixing/optimization based on user feedback and known issues, Week 10 final fixes/optimization, demo preparation, presentation/poster preparation, Practical UX Review documentation, Manual MVP Workflow Testing documentation, existing-flow Playwright validation, security review documentation, future hardening backlog documentation, known limitation documentation, and work log completion.

This SRS defines product requirements only. Detailed test results, Practical UX Review checklist items, Manual MVP Workflow Testing details, automated test coverage, security review details, security backlog cards, user feedback session notes, and detailed validation results are tracked in the Test Planning Document.

Current work log status:

- 75.85 hours worked
- 24.15 hours remaining toward the 100-hour capstone requirement

---

2. System Overview

System Name: TaskAnchor

Purpose:

Provide a structured system that ensures users revisit, track, and complete real-life priorities over time.

Core Problem:

Users forget, ignore, or inconsistently manage tasks because current tools do not enforce follow-up or visibility of incomplete work.

System Positioning:

A personal follow-through system that keeps unfinished work visible and actionable until completed, while preserving context and next steps.

---

3. Scope

In Scope (MVP)

- Basic user authentication
- Create tasks
- Edit tasks
- Delete tasks
- View active tasks
- Task status tracking (Active, InProgress, Completed)

- Optional due dates
- Optional priority levels
- Overdue task identification (derived)
- Add Progress Log entries
- View Progress Log entries
- NextAction tracking
- Task sorting
- Angular frontend (Register, Login, Task List, Task Create)
- Basic required-field validation for existing forms
- Backend required-field validation for existing task/authentication inputs selected during validation hardening
- Backend maximum length validation for selected existing text fields
- Practical clarity/access corrections for existing MVP screens
- Confirmation before terminal Completed status
- Confirmation before destructive Delete Task action
- User-facing status text readability for existing status display
- MVP-level suspicious-input validation for existing authentication behavior
- MVP-level password hashing behavior
- MVP-level stored-text display regression coverage for existing Task List fields
- Selected security source-pattern review documentation for current MVP limitations
- Selected API request binding hardening for Create Task and Update Task
- Security review documentation for current MVP limitations
- Future security hardening backlog documentation
- Course-required user feedback documentation if feedback affects known limitations, future enhancements, requirements wording, or feature-list wording

Current MVP implementation uses a simplified single-user approach.

Out of Scope (MVP)

- New MVP features
- New workflows
- Time-specific due dates or due times
- Subtasks
- Task archiving/restoration
- Reactivation of completed tasks
- Categories/tags
- Multi-user features
- Notifications/reminders
- AI features
- Payments
- Advanced prioritization
- UX redesign
- UI polish beyond basic clarity and access
- CAPTCHA, rate limiting, lockouts, account verification, or new authentication workflows
- Token/session enforcement beyond current MVP scope
- Multi-user authorization or ownership enforcement beyond current MVP scope
- Role-based access control
- Navbar/logout/session guard behavior
- CSRF protection for future browser-session authentication
- Production-grade dependency vulnerability remediation process

- Forced breaking Angular major-version dependency remediation during the current MVP documentation phase

- Production deployment security hardening

- Progress Log edit/delete/revision-history behavior

- File upload, attachment upload, task import, backup restore, file export, generated-file, or bulk serialized-data workflows

- Broad DRY refactoring or architecture cleanup during post-MVP validation

- Automatically adding user-requested features from Week 8 user feedback without separate scope approval

Security scope boundary:

- The MVP includes basic authentication, password hashing, required-field validation for current auth forms, selected backend validation hardening for existing auth/task/progress-log inputs, suspicious-input validation for existing authentication behavior, regression coverage for script-like stored text display in existing Task List fields, selected source-pattern review documentation, and selected API request binding hardening for Create Task and Update Task.

- The MVP does not claim production-grade authentication or authorization.

- Token/session enforcement, multi-user ownership checks, rate limiting, lockouts, CAPTCHA, account verification, CSRF protection, logout/session guards, and role-based access control are future hardening items.

- Security review findings should be documented as known limitations or future enhancements unless a small existing-behavior defect is explicitly selected for correction.

Course check-in scope boundary:

- Week 8 user feedback testing may identify bugs, confusion points, documentation updates, known limitations, or future enhancement ideas.

- User feedback does not automatically expand MVP scope.

- Confirmed bugs or small existing-flow clarity/access corrections may be considered during Week 9 bug fixing and optimization.

- Feature requests from user feedback should be documented as future enhancements unless explicitly selected later and kept within approved scope.

---

4. Stakeholders

Primary User:

Single individual managing personal tasks and responsibilities.

Course / Review Stakeholders:

Instructor, evaluator, and user feedback participant who may review or test the current MVP for capstone validation and final presentation preparation.

---

5. Functional Requirements

5.1 Authentication

User can:

- Register
- Log in

Behavior:

- Passwords are hashed

- Login navigates to task list

- Basic authentication only

- Token enforcement is not required for MVP

- Register requires email and password

- Login requires email and password

- Blank or whitespace email/password input displays visible validation feedback in the frontend

- Blank or whitespace email/password input does not submit Login/Register requests from the frontend

- Backend Login rejects blank or whitespace email/password input with BadRequest("Email and password are required.")

- Failed Login displays visible error feedback

- Failed Register displays visible error feedback

- Duplicate Register email returns BadRequest("Email is already registered.") and does not save a second user

- Register email values over 255 characters return BadRequest("Email must be 255 characters or fewer.") and are not saved

- Login email values over 255 characters return BadRequest("Email must be 255 characters or fewer.") before authentication lookup

- Register password values over 128 characters return BadRequest("Password must be 128 characters or fewer.") and are not saved

- Login password values over 128 characters return BadRequest("Password must be 128 characters or fewer.") before password comparison

- Clearly invalid Register email format returns BadRequest("Email format is invalid.") and is not saved

- Script-like Register email input is rejected as invalid email format and is not saved

- SQL-like Register password input remains plain password input and is stored only as a hash when paired with a valid email

- Script-like or SQL-like Login input must not bypass backend authentication behavior

- Authentication behavior is MVP-level and does not include production token/session enforcement

---

5.2 Task Management

User can:

- Create task
- Edit task
- Delete task
- View active task list

Task fields:

- TaskId
- Title
- Description
- Status (Active, InProgress, Completed)
- PriorityLevel (Low, Medium, High)
- DueDate (optional)
- NextAction (optional)
- LastUpdatedDate

Behavior:

- Tasks remain visible until completed
- Completed tasks are excluded from the active task list
- Task edits persist through backend update behavior
- Existing task fields can be edited through the task edit flow where exposed in the MVP UI
- Create Task requires Title
- Create Task API rejects blank or whitespace-only Title values with BadRequest("Title is required.") and does not save invalid tasks
- Create Task API rejects Title values over 100 characters with BadRequest("Title must be 100 characters or fewer.") and does not save invalid tasks
- Create Task API rejects Description values over 500 characters with BadRequest("Description must be 500 characters or fewer.") and does not save invalid tasks
- Create Task API rejects NextAction values over 200 characters with BadRequest("Next Action must be 200 characters or fewer.") and does not save invalid tasks
- Create Task request binding uses a narrow CreateTaskRequest shape for editable create fields rather than binding directly to TaskItem
- Create Task server-controlled fields are assigned by backend/default behavior
- Update Task request binding uses a narrow UpdateTaskRequest shape for editable update fields rather than binding directly to TaskItem
- Blank or whitespace Title displays visible validation feedback in the frontend
- Blank or whitespace Title does not submit a create request from the frontend
- Description is supported as an existing Task field
- Description is captured during task creation
- Description is displayed in the active task list when task data includes description
- Description is editable through the existing task edit flow

- PriorityLevel is captured during task creation and editable through the existing task edit flow

- DueDate is captured during task creation and editable through the existing task edit flow as a date-only value

- DueDate is displayed in the active task list when task data includes a due date

- DueDate edit/save persists through backend update behavior and remains visible after browser refresh

- NextAction is captured during task creation and editable through the existing task edit flow

- Script-like and SQL-like task input values are treated as plain text data

- Delete Task requires confirmation before the destructive delete request is submitted

- Delete confirmation identifies the clicked task by title

- Cancelling Delete keeps the task and does not submit a delete request

- Accepting Delete submits the delete request and refreshes the active task list

Security-related MVP boundary:

- The current MVP uses simplified single-user task behavior.

- Future versions should enforce authenticated task API access and record ownership before production use.

- Multi-user task ownership enforcement is outside MVP scope.

5.3 Status Tracking

Task states:

- Active

- InProgress
- Completed

Behavior:

- Active tasks can move to InProgress
- InProgress tasks can move to Completed
- Completed is terminal
- Completed tasks do not reactivate in the MVP
- A confirmation is shown before moving a task to Completed
- Cancelling completion keeps the task active and does not complete the task
- User-facing display text for InProgress is shown as "In Progress" for readability
- Backend/status values remain InProgress

Completed tasks are excluded from the active list.

5.4 Priority and Sorting

Tasks are ordered by:

1. Overdue tasks
2. Due date ascending
3. No due date → priority High → Low

PriorityLevel behavior:

- PriorityLevel supports Low, Medium, and High
- PriorityLevel is captured during task creation
- PriorityLevel displays in the active task list
- PriorityLevel can be edited through the existing task edit flow
- Advanced prioritization is outside MVP scope

5.5 Due Date and Overdue Behavior

DueDate is an optional date-only MVP field.

A task is overdue if:

- Status is Active or InProgress
- DueDate exists
- DueDate is before the current date

Overdue is derived, not stored.

Completed tasks are not overdue.

DueDate behavior:

- DueDate can be selected during task creation
- DueDate can be edited through the existing task edit flow

- DueDate edit/save persists after browser refresh
- Blank DueDate is treated as null
- DueDate remains date-only in the MVP

Time-specific due dates or due times, such as "due at 5:00 PM," are outside MVP scope and are future enhancement candidates only.

5.6 Progress Log

Users can:

- Add Progress Log entries
- View Progress Log entries

Each entry includes:

- EntryText
- CreatedDate

Behavior:

- Entries are tied to a task
- Entries are persisted via backend
- Entries are displayed in UI per task
- Entries support resuming work and tracking history

- Blank or whitespace Progress Log entries are not saved

- Blank or whitespace Progress Log save exits the current Progress Log input state without creating an empty entry

- Saved Progress Log EntryText is trimmed before persistence

- Script-like Progress Log EntryText is stored as plain text

- Progress Log EntryText values over 500 characters return BadRequest("Entry Text must be 500 characters or fewer.") and are not saved

Security-related MVP boundary:

- Current Progress Log behavior is part of the simplified single-user MVP.

- Future versions should enforce authenticated access and parent-task ownership checks for Progress Log endpoints before production use.

Progress Log edit, delete, and revision history behavior is outside MVP scope.

5.7 NextAction

NextAction is an optional field per task.

Behavior:

- Represents the next concrete step

- Captured during task creation in the frontend as nextAction

- Displayed in task list when present

- Editable through the existing task edit flow

- Referred to in SRS/API terminology as NextAction

- NextAction values over 200 characters are rejected by backend Create Task validation

- Script-like NextAction display text is rendered as plain text in the existing Task List display

NextAction is an existing MVP field and does not represent a new workflow.

5.8 Course User Feedback Requirement

Week 8 course work requires a user feedback session using the current TaskAnchor MVP.

Behavior:

- A person other than the developer should use the current TaskAnchor MVP.

- The tester should be observed without being led through the workflow unless a hint is necessary.

- Testing notes should record confusion points, bugs, hesitation, and any hints given.

- Follow-up feedback should be collected about usefulness, confusing areas, what worked well, desired features, and what the tester would change.

- Testing notes should be saved in the repository.

- Feedback should be triaged into confirmed bugs, small clarity/access corrections, known limitations, or future enhancement ideas.

- SRS changelog and feature-list wording should be updated if user feedback affects requirements, known limitations, or future enhancement notes.

Scope boundary:

- User feedback does not automatically create new MVP features.

- Week 9 bug fixing/optimization should be based on confirmed bugs, known issues, or small existing-flow clarity/access corrections.

- Week 10 final fixes/optimization should prioritize stability and final project readiness.

---

6. Non-Functional Requirements

6.1 Security

- Password hashing implemented

- Basic authentication only

- Token enforcement is not required for MVP

- Login/Register input is sent through structured JSON request bodies

- Backend authentication validation includes selected required-field, length-limit, duplicate email, and Register email-format checks

- Script-like or SQL-like Login input must not bypass backend authentication checks

- Script-like Register email input is rejected as invalid email format

- SQL-like Register password input is stored only as a hash when Register email is valid

- Raw passwords must not be stored

- Existing authentication suspicious-input handling is validated for the MVP

- Current reviewed backend behavior uses Entity Framework Core/LINQ patterns rather than identified string-concatenated dynamic SQL in reviewed snippets

- No confirmed SQL injection or command injection vulnerability was identified during the current review of provided code snippets

- Script-like task and Progress Log values are treated as plain text data

- Angular text interpolation should remain preferred for displaying user-entered text

- Stored-text display regression coverage confirms script-like Title, Description, NextAction, and Progress Log text render as text without creating script elements in the existing Task List display

- Unsafe dynamic execution pattern review coverage is documented in the Test Planning Document

- File upload exposure review coverage is documented in the Test Planning Document

- Path traversal exposure review coverage is documented in the Test Planning Document

- Unsafe deserialization exposure review coverage is documented in the Test Planning Document

- CORS configuration review coverage is documented in the Test Planning Document

- Over-posting/mass-assignment request binding review coverage is documented in the Test Planning Document

- Create Task and Update Task request binding use narrow request DTOs for editable fields only

- Unsafe dynamic HTML patterns, direct DOM manipulation, and dynamic command/query construction should be avoided in future changes

- The MVP uses simplified single-user behavior and does not provide production-grade authorization

- Full production security hardening, including CAPTCHA, lockouts, rate limiting, account verification, token/session enforcement, role-based access control, multi-user ownership enforcement, CSRF protection, and production deployment hardening, is outside MVP scope

---

6.2 Usability

- Minimal UI
- Tasks remain visible until completed
- NextAction visible when present
- Progress Log entries visible per task
- Forms use ngModel for state-driven binding
- Changes reflected through refresh pattern
- Practical clarity and access are allowed for existing MVP screens
- UX redesign and UI polish beyond basic clarity/access are out of scope
- Existing form validation messages should be visible and understandable
- Required-field validation should preserve user-entered values so the user can correct the form
- Error/validation styling should remain consistent across existing Login, Register, and Create Task screens
- Destructive Delete Task behavior should require confirmation before submission
- Delete confirmation should identify the selected task by title
- User-facing InProgress text should display as "In Progress" for readability while preserving internal/backend status values
- Week 8 user feedback may identify confusing areas that should be documented or triaged for small clarity/access fixes if they remain within MVP scope

---

6.3 Reliability

- Data persisted in SQL Server

- No data loss under normal local development usage

- Task updates use backend persistence and frontend refresh behavior

- Frontend refresh behavior is used after successful create, edit, delete, status update, and Progress Log save operations

- Cancelled Delete actions do not call deleteTask or refresh the task list

- Accepted Delete actions call deleteTask and refresh the task list after success

- Backend validation rejects selected invalid or oversized existing text inputs before saving

- Invalid task/auth/progress-log validation failures do not persist invalid records

- Week 9 and Week 10 fixes should prioritize stability and final project readiness over new feature work

---

7. System Architecture

Frontend: Angular

Backend: ASP.NET Core API

Database: SQL Server

ORM: Entity Framework Core

Frontend implementation status:

- RegisterComponent implemented

- LoginComponent implemented

- TaskListComponent implemented

- TaskFormComponent implemented

- Routing implemented:

  - / → LoginComponent

- /register → RegisterComponent
- /tasks → TaskListComponent

Backend implementation status:

- Authentication endpoints implemented
- Task CRUD endpoints implemented
- Status update endpoint implemented
- Progress Log endpoints implemented
- Core rule services implemented
- Backend validation hardening implemented for selected existing task, progress-log, and authentication fields
- Create Task request binding uses CreateTaskRequest
- Update Task request binding uses UpdateTaskRequest

Security architecture boundary:

- Current MVP authentication is basic.
- Current MVP does not include token/session enforcement.
- Current MVP does not include multi-user authorization or record ownership enforcement.
- Future architecture hardening should enforce authenticated access, record ownership, and deny-by-default authorization before production use.

---

8. Data Model

AppUser

- UserId
- Email
- PasswordHash

Task

- TaskId
- UserId (FK)
- Title
- Description
- Status
- PriorityLevel
- DueDate
- NextAction
- LastUpdatedDate

ProgressLogEntry

- ProgressLogEntryId
- TaskId (FK)
- EntryText
- CreatedDate

Frontend Task objects use the full returned/rendered Task shape:

- taskId
- title

- description
- status
- priorityLevel
- dueDate
- nextAction
- lastUpdatedDate

Frontend Task objects may also include:

- progressLogs

Frontend ProgressLog display objects support:

- text
- entryText

Create payloads are smaller than returned/rendered Task objects and include:

- title
- description
- priorityLevel
- dueDate
- nextAction

Create payload runtime behavior:

- priorityLevel is mapped to backend enum-compatible values

- Blank dueDate is sent as null
- dueDate is treated as a date-only value in the MVP
- status is not supplied by the frontend Create Task form
- status is assigned by backend/default create behavior

Backend Create Task request shape includes editable task fields only:

- Title
- Description
- PriorityLevel
- DueDate
- NextAction

Backend Update Task request shape includes editable task fields only:

- Title
- Description
- PriorityLevel
- DueDate
- NextAction

Security data boundary:

- UserId exists in the backend data model.
- Current MVP uses simplified single-user behavior.
- Future multi-user versions should enforce authenticated ownership so one user cannot access or modify another user's task or Progress Log data.

---

9. Core Logic

Revisit Rule:

Tasks remain visible until completed.

InProgress Rule:

Tasks can move from Active to InProgress as work begins.

Status Display Rule:

The internal/backend status value remains InProgress, but user-facing display text uses "In Progress" for readability.

Terminal Completion Rule:

Completed tasks are terminal and excluded from the active task list.

Completion Confirmation Rule:

The user receives confirmation before a task is moved to Completed.

Delete Confirmation Rule:

The user receives title-specific confirmation before a task is deleted. Cancelling Delete prevents deleteTask from running. Accepting Delete allows the existing delete and refresh behavior to proceed.

Overdue Rule:

Overdue is derived dynamically and not stored.

Date-Only DueDate Rule:

DueDate is treated as a date-only MVP field. Due times and time-specific deadlines are not part of the MVP.

Required Title Rule:

A task requires a non-blank Title before the frontend submits a create request. The backend also rejects blank or whitespace-only Title values.

Task Text Length Rule:

Backend Create Task validation rejects Title values over 100 characters, Description values over 500 characters, and NextAction values over 200 characters.

Task Request Binding Rule:

Create Task and Update Task accept narrow backend request DTOs for editable task fields rather than binding directly to the TaskItem entity.

PriorityLevel Rule:

PriorityLevel is stored, displayed, and editable as an existing MVP field.

NextAction Rule:

Optional next step is stored, displayed when present, and editable through the existing task edit flow.

Progress Log Rule:

Entries are appended and displayed per task. Blank or whitespace Progress Log entries are not saved. Saved EntryText is trimmed. EntryText values over 500 characters are rejected.

Authentication Required-Field Rule:

Login and Register require non-blank email and password before the frontend submits authentication requests. Backend Login and Register also reject blank or whitespace email/password values.

Authentication Length Rule:

Register and Login reject email values over 255 characters and password values over 128 characters.

Register Duplicate Email Rule:

Register rejects duplicate email addresses and does not save a second user.

Register Email Format Rule:

Register rejects clearly invalid email format and script-like email input with BadRequest("Email format is invalid.").

Password Hashing Rule:

Registered passwords are stored as hashes, not raw password text.

Suspicious Authentication Input Rule:

Script-like or SQL-like Login input is treated as plain text and must not bypass backend authentication behavior. SQL-like Register password input is treated as plain password input and is stored only as a hash when paired with a valid email.

Stored Text Display Rule:

Script-like task Title, Description, NextAction, and Progress Log values should render as plain text in the Task List and must not create script elements.

Sorting Rule:

Tasks sort by:

1. Overdue
2. DueDate ascending
3. No due date → priority High → Low

Update Tracking Rule:

LastUpdatedDate is updated on:

- create
- edit
- status change
- Progress Log entry

Refresh Rule:

Frontend updates use the centralized refresh pattern through refreshTasks().

User Feedback Triage Rule:

Week 8 user feedback should be triaged into confirmed bugs, small existing-flow clarity/access corrections, known limitations, or future enhancements. Feedback should not automatically expand MVP scope.

Future Authorization Rule:

If TaskAnchor is expanded beyond the simplified single-user MVP, server-side authorization should enforce authenticated access and record ownership for task and Progress Log data.

---

10. Constraints & Risks

- Time constraint: 100-hour capstone
- Risk of scope creep
- Need strict frontend/backend alignment
- Basic authentication only
- Avoid overengineering Angular
- Maintain Task object shape consistency in frontend tests
- Keep overdue derived, not stored
- Keep DueDate date-only in the MVP
- Keep due times and time-specific deadlines out of MVP scope
- Keep UX work limited to practical clarity/access for existing MVP flows only
- Avoid adding new workflows during post-MVP validation
- Keep authentication validation limited to current MVP Login/Register behavior
- Full production security hardening is outside MVP scope
- Remaining interaction-mode overlap issues should be documented as known limitations unless selected for a future small correction
- Avoid broad DRY refactoring during validation unless required by a defect
- MVP access control is intentionally limited by the simplified single-user approach
- Token/session enforcement is outside MVP scope
- Multi-user ownership enforcement is outside MVP scope
- Current MVP should not be represented as production-grade security
- Future deployment would require additional authentication, authorization, ownership, dependency, and configuration hardening
- Security backlog items should not interrupt final capstone documentation, demo preparation, poster preparation, presentation preparation, or work log completion unless a small existing-behavior defect is explicitly selected
- Week 8 user feedback may identify bugs or confusion not currently covered by automated tests

- User feedback should be triaged before any changes are made

- Week 9 and Week 10 bug fixing should prioritize stable MVP behavior and assignment deliverables over new feature work

- Feature requests from user testing should be documented as future enhancements unless they are explicitly selected later within approved scope

---

11. Future Enhancements

The following are future ideas only and are not part of the MVP:

- Subtasks
- Archiving
- Reactivation
- Notifications
- Multi-user support
- Task assignment
- Advanced prioritization
- AI features
- Time-specific due dates or due times
- Date/time deadline behavior
- CAPTCHA
- Rate limiting
- Account lockout
- Account verification
- Token/session enforcement

- Logout/session guard behavior

- Role-based access control

- Server-side authorization enforcement for task APIs

- Authenticated record ownership checks for tasks

- Authenticated record ownership checks for Progress Log endpoints

- Deny-by-default authorization policy

- CSRF review/protection for future browser-session authentication

- Production CORS/deployment review if the app is deployed beyond local capstone/demo use

- Access-control logging for rejected protected requests

- Secure session/token lifetime review

- Breaking Angular dependency remediation requiring planned major-version upgrade review

- File upload controls if attachment/import/profile-image/task-file features are added later

- Path traversal controls if file import/export/download/generated-file features are added later

- Unsafe deserialization controls if import, backup restore, bulk API, or complex serialized-data exchange features are added later

- Progress Log edit/delete/revision history

- Interaction-mode guardrails for edit mode and Progress Log mode

- Broader visual distinction between Create Task and Task List sections

- Broad DRY refactoring/code cleanup

- Feature ideas identified during Week 8 user feedback that are outside current MVP scope

---

12. Use Cases (MVP)

UC-1: Register User

UC-2: Login

UC-3: Create Task

UC-4: Edit Task

UC-5: Delete Task

UC-6: Start Task (Active → InProgress)

UC-7: Complete Task

UC-8: View Active Tasks

UC-9: Set Priority

UC-10: Set Due Date

UC-11: Add Progress Log

UC-12: View Progress Logs

UC-13: Set/View NextAction

UC-14: Validate Required Form Fields

UC-15: Display Authentication/Create Task Validation Feedback

UC-16: Confirm Destructive Delete Task Action

UC-17: Collect User Feedback for Course Check-In

---

13. External Interface (REST API)

Auth:

- POST /api/auth/register
- POST /api/auth/login

Tasks:

- GET /api/tasks
- POST /api/tasks
- PUT /api/tasks/{id}
- PUT /api/tasks/{id}/status
- DELETE /api/tasks/{id}

Progress Log:

- GET /api/tasks/{id}/progress
- POST /api/tasks/{id}/progress

Security API boundary:

- Current MVP API behavior supports the simplified single-user capstone implementation.
- Future production-ready versions should require authenticated access for protected task and Progress Log endpoints.
- Future production-ready versions should enforce ownership checks for task and Progress Log records.

---

14. Remaining Course Check-In Alignment

MVP feature development is complete, but course deliverables remain.

Week 8 user feedback testing:

- A person other than the developer should use the current TaskAnchor MVP.

- Testing notes should record user confusion, hesitation, bugs, hints, and follow-up answers.

- Feedback should be saved in the repository.

- SRS changelog and feature-list wording should be updated if feedback affects requirements, known limitations, or future enhancement notes.

Week 9 bug fixing and optimization:

- Work should focus on confirmed bugs, known issues, optimization, and small existing-flow clarity/access corrections if needed.

- New MVP features and new workflows should remain out of scope.

Week 10 final fixes and optimization:

- Work should focus on final stability, final fixes, final documentation alignment, and final project readiness.

Final presentation:

- Presentation materials should describe the problem, solution, planning/development process, technology choices, user testing results, what changed because of user feedback, what worked well, what could be improved, future direction, and the live MVP demo.

---

15. MVP Completion Status

MVP feature development is complete.

Completed MVP areas:

- Backend authentication
- Frontend authentication
- Login/Register required-field validation
- Backend Login blank-input validation
- Login/Register visible validation/error feedback
- Password hashing
- Authentication suspicious-input handling validation
- Duplicate Register email handling
- Register invalid email format validation
- Register/Login email length validation
- Register/Login password length validation
- Task CRUD
- Delete Task confirmation
- Create Task required Title validation
- Backend Create Task title validation
- Backend Create Task text length validation for Title, Description, and NextAction
- Backend Progress Log EntryText length validation
- Create Task Description support
- Create Task request binding through CreateTaskRequest
- Update Task request binding through UpdateTaskRequest
- Active task list
- Status transitions
- User-facing "In Progress" status readability
- Completed task exclusion
- Completion confirmation before Completed
- DueDate support as a date-only field
- DueDate edit support as a date-only value

- DueDate edit/save persistence after browser refresh
- Description display with task data
- Due Date display with task data
- PriorityLevel support
- PriorityLevel display/edit behavior
- Derived overdue logic
- Sorting
- NextAction creation/display/edit behavior
- Progress Log add/view behavior
- Progress Log EntryText trim behavior
- Progress Log script-like plain-text handling
- Blank Progress Log save handling
- Angular frontend integration
- Refresh-based frontend update pattern
- Existing-flow Playwright validation for documented E2E gaps
- Manual MVP Workflow Testing confirmation
- Full Practical UX Review confirmation with remaining limitations documented in the Test Planning Document
- Security review completed for Injection and Broken Access Control study topics
- Backend validation/test-hardening completed for selected Injection-review backlog cards
- Security source-pattern review coverage completed for selected current-source exposure areas
- CORS configuration review coverage completed
- Over-posting/mass-assignment request binding review completed for selected API request shapes
- Stored-text display regression coverage completed for Task List Title, Description, NextAction, and Progress Log text

- Future security hardening backlog documented in the Test Planning Document

Latest confirmed automated validation totals:

- Backend xUnit: 55 of 55 tests passed
- Angular Karma: 159 of 159 tests passed
- Playwright: 36 passed, 0 skipped, 0 failed

Remaining capstone work is not new feature development. Remaining work includes Week 8 user feedback testing, Week 9 bug fixing/optimization based on feedback and known issues, Week 10 final fixes/optimization, documentation alignment, demo preparation, presentation/poster preparation, security review documentation, known limitation documentation, future backlog documentation, and work log completion.

Detailed Practical UX Review, Manual MVP Workflow Testing, automated test results, security review details, security backlog cards, user feedback notes, and test coverage status are tracked in the Test Planning Document, not in the SRS.

---

16. Changelog

v1.16 - 05/16/2026

- Updated SRS version to v1.16 and date to 05/16/2026.
- Updated current work log status to 75.85 hours worked and 24.15 hours remaining toward the 100-hour capstone requirement.
- Updated remaining capstone work to include Week 8 user feedback testing, Week 9 bug fixing/optimization, Week 10 final fixes/optimization, and final presentation preparation.

- Added Course Check-In scope boundary explaining that user feedback may produce bugs, clarity/access corrections, known limitations, documentation updates, or future enhancement ideas, but does not automatically expand MVP scope.

- Added Course User Feedback Requirement under Functional Requirements.

- Added User Feedback Triage Rule to Core Logic.

- Added Remaining Course Check-In Alignment section.

- Added UC-17: Collect User Feedback for Course Check-In.

- Added Week 8 user feedback testing, Week 9 bug fixing/optimization, Week 10 final fixes/optimization, and final presentation alignment to MVP remaining work.

- Added selected security source-pattern review documentation to MVP scope.

- Added selected API request binding hardening for Create Task and Update Task to MVP scope.

- Documented CreateTaskRequest and UpdateTaskRequest request binding hardening.

- Documented Create Task and Update Task request binding as narrow request DTOs rather than direct TaskItem binding.

- Documented selected security source-pattern review coverage in Non-Functional Security requirements.

- Updated Future Enhancements to move completed review items out of future-only wording and preserve future controls for file upload, path traversal, unsafe deserialization, production CORS/deployment review, and breaking Angular dependency remediation.

- Updated MVP Completion Status with completed security source-pattern review coverage, CORS configuration review coverage, and over-posting/mass-assignment request binding review.

- Updated latest automated validation totals:

  - Backend xUnit: 55 of 55 tests passed.

  - Angular Karma: 159 of 159 tests passed.

  - Playwright: 36 passed, 0 skipped, 0 failed.

- Maintained scope boundaries: no new MVP features, no new workflows, no token/session enforcement, no multi-user ownership enforcement, no CAPTCHA, no rate limiting, no lockouts, no account verification, no logout/session guard behavior, no CSRF implementation, no role-based access control, no Progress Log edit/delete/revision history, no due times, no forced breaking Angular upgrade, and no broad refactoring.

v1.15 - 05/14/2026

- Updated SRS version to v1.15 and date to 05/14/2026.

- Consolidated completed Week 7 validation-hardening and security regression work from the documentation update packet.

- Completed Backlog Card 2 for task/progress-log text fields:

  - Added backend Create Task Title length validation.

  - Added backend Create Task Description length validation.

  - Added backend Create Task NextAction length validation.

  - Added backend Progress Log EntryText length validation.

  - Documented Title max length as 100 characters.

  - Documented Description max length as 500 characters.

  - Documented NextAction max length as 200 characters.

  - Documented Progress Log EntryText max length as 500 characters.

- Completed Backlog Card 3: Add backend Progress Log input normalization review.

  - Documented Progress Log EntryText trim behavior.

  - Documented script-like Progress Log EntryText as stored plain text.

- Completed Backlog Card 6: Add direct API tests for suspicious task input.

  - Documented script-like and SQL-like task input values as plain text data.

- Completed Backlog Card 26: Add backend authentication input length validation.

  - Documented Register/Login email max length as 255 characters.

  - Documented Register/Login password max length as 128 characters.

- Completed Backlog Card 8: Add duplicate and invalid email validation review.

  - Documented backend Login blank-input validation.

  - Documented duplicate Register email rejection.

  - Documented valid Register and valid Login regression coverage as completed in the Test Planning Document.

  - Documented Register invalid email format validation.

  - Documented script-like Register email rejection.

  - Documented SQL-like Register password input as hashed when paired with a valid email.

- Completed Backlog Card 4: Add stored XSS regression tests for task display fields.

  - Documented script-like Task List Title display as plain text.

  - Documented script-like Task List Description display as plain text.

  - Documented script-like Task List NextAction display as plain text.

  - Documented script-like Progress Log display as plain text.

  - Documented that no script elements are created by these existing Task List display fields.

- Updated Security requirements to reflect completed backend validation hardening and frontend stored-text display regression coverage.

- Updated Core Logic with Task Text Length Rule, Authentication Length Rule, Register Duplicate Email Rule, Register Email Format Rule, and Stored Text Display Rule.

- Updated Data Model to note frontend ProgressLog display objects support text and entryText.

- Updated Future Enhancements to remove completed items from future-only status and retain remaining future hardening items:

  - Unsafe dynamic execution pattern review.

  - Dependency vulnerability review.

  - File upload exposure review before adding file features.

  - Path traversal review before adding file/import/export features.

  - Deserialization safety review before adding import/API bulk features.

  - Production-grade authentication and authorization hardening.

- Updated MVP Completion Status with completed Week 7 validation/test-hardening cards.

- Updated latest automated validation totals:

  - Backend xUnit: 45 of 45 tests passed.

  - Angular Karma: 159 of 159 tests passed.

  - Playwright: 36 passed, 0 skipped, 0 failed.

- Maintained scope boundaries: no new MVP features, no new workflows, no token/session enforcement, no multi-user ownership enforcement, no CAPTCHA, no rate limiting, no lockouts, no account verification, no logout/session guard behavior, no CSRF implementation, no role-based access control, no Progress Log edit/delete/revision history, no due times, and no broad refactoring.

v1.14 - 05/13/2026

- Completed Backlog Card 1: Add backend Title validation for Create Task.

- Added backend validation so blank or whitespace-only Create Task titles return BadRequest("Title is required.") and do not save a task.

- Added backend xUnit coverage for blank Create Task title rejection.

- Updated backend validation result from 27/27 passing tests to 28/28 passing tests.

- Confirmed this is a defect correction and validation-hardening update for existing MVP behavior, not a new workflow.

v1.13 (05/06/2026)

- Updated SRS version to v1.13 and date to 05/06/2026.

- Updated current work log status to 66.48 hours worked and 33.52 hours remaining toward the 100-hour capstone requirement.

- Updated remaining capstone work to include security review documentation, future hardening backlog documentation, and known limitation documentation.

- Clarified that detailed security review results and security backlog cards belong in the Test Planning Document, not the SRS.

- Added MVP-level suspicious-input validation and password hashing behavior to In Scope.

- Added security review documentation and future security hardening backlog documentation to In Scope.

- Added multi-user authorization or ownership enforcement beyond current MVP scope to Out of Scope.

- Added CSRF protection for future browser-session authentication, production-grade dependency vulnerability remediation, and production deployment security hardening to Out of Scope.

- Added Security scope boundary clarifying that MVP security behavior is basic and not production-grade.

- Updated Authentication requirements to clarify that authentication behavior is MVP-level and does not include production token/session enforcement.

- Added Task Management security boundary explaining simplified single-user behavior and future ownership enforcement needs.

- Added Progress Log security boundary explaining future authenticated access and parent-task ownership checks.

- Updated Non-Functional Security requirements to document current security review conclusions:

  - Password hashing is implemented.

  - Suspicious authentication input is validated for MVP behavior.

  - Reviewed backend behavior uses Entity Framework Core/LINQ patterns rather than identified string-concatenated dynamic SQL in reviewed snippets.

  - No confirmed SQL injection or command injection vulnerability was identified during the current review of provided code snippets.

  - Angular text interpolation should remain preferred for displaying user-entered text.

  - Unsafe dynamic HTML patterns, direct DOM manipulation, and dynamic command/query construction should be avoided in future changes.

  - Production-grade authorization remains outside MVP scope.

- Added Security architecture boundary under System Architecture.

- Added Security data boundary under Data Model.

- Added Suspicious Authentication Input Rule to Core Logic.

- Added Future Authorization Rule to Core Logic.

- Updated Constraints & Risks to include limited MVP access control, no token/session enforcement, no multi-user ownership enforcement, no production-grade security claim, and future deployment hardening needs.

- Updated Future Enhancements with security hardening items:

  - Logout/session guard behavior.

  - Server-side authorization enforcement for task APIs.

  - Authenticated record ownership checks for tasks.

  - Authenticated record ownership checks for Progress Log endpoints.

  - Deny-by-default authorization policy.

  - CSRF review/protection for future browser-session authentication.

  - CORS configuration review before deployment.

  - Access-control logging for rejected protected requests.

  - Secure session/token lifetime review.

  - Backend Title validation parity with frontend Create Task validation.

  - Backend length limits for text fields.

  - Stored XSS regression tests for displayed task and Progress Log fields.

  - Dependency vulnerability review before final release or deployment.

- Added Security API boundary under External Interface.

- Updated MVP Completion Status to include completed security review for Injection and Broken Access Control study topics.

- Updated MVP Completion Status to clarify that future security hardening backlog is documented in the Test Planning Document.

- Maintained scope boundaries: no new MVP features, no new workflows, no token/session enforcement, no multi-user ownership enforcement, no CAPTCHA, no rate limiting, no lockouts, no account verification, no logout/session guard behavior, no CSRF implementation, no role-based access control, no Progress Log edit/delete/revision history, no due times, and no broad refactoring.

v1.12 (05/05/2026)

- Updated SRS version to v1.12 and date to 05/05/2026.

- Clarified current remaining capstone work after Playwright expansion and Practical UX Review confirmation.

- Added Delete Task confirmation to MVP scope as an existing destructive-action safety correction.

- Added title-specific Delete confirmation behavior to Task Management.

- Added Delete Confirmation Rule to Core Logic.

- Added Delete Task confirmation to Usability and Reliability requirements.

- Added UC-16 for Confirm Destructive Delete Task Action.

- Documented that cancelling Delete prevents the delete request and accepting Delete proceeds with existing delete and refresh behavior.

- Added user-facing status text readability behavior for InProgress.

- Clarified that internal/backend status values remain InProgress while user-facing text displays "In Progress."

- Added Status Display Rule to Core Logic.

- Added user-facing "In Progress" status readability to MVP Completion Status.

- Added Description display and Due Date display behavior to Task Management and MVP Completion Status.

- Added DueDate edit/save persistence after browser refresh to Due Date behavior and MVP Completion Status.

- Updated Future Enhancements to include interaction-mode guardrails, broader section visual distinction, and broad DRY cleanup as future/non-MVP items.

- Added broad DRY refactoring to out-of-scope items during post-MVP validation.

- Clarified remaining interaction-mode overlap issues should be documented as known limitations unless selected for future correction.

- Updated MVP Completion Status to include existing-flow Playwright validation, Manual MVP Workflow Testing confirmation, and full Practical UX Review confirmation.

- Kept detailed automated test results, Manual MVP Workflow Testing checklists, and Practical UX Review findings in the Test Planning Document, not in the SRS.

- Maintained scope boundaries: no new MVP features, no new workflows, no completed-task restore/reactivation/archive behavior, no due times, no Progress Log edit/delete/revision history, and no broad refactoring.

v1.11 (05/04/2026)

- Added basic required-field validation behavior for existing Login, Register, and Create Task forms.

- Clarified that Login and Register require non-blank email and password before frontend auth requests are submitted.

- Clarified that Create Task requires a non-blank Title before frontend create requests are submitted.

- Added validation feedback expectations for existing forms.

- Added suspicious-input authentication handling to MVP validation scope.

- Clarified that script-like or SQL-like authentication input must not bypass backend authentication behavior.

- Added password hashing behavior to Security and Core Logic sections.

- Added production security hardening items to out-of-scope and future enhancement sections.

- Added Description to Create Task payload documentation.

- Clarified that Create Task status is backend/default behavior and is not a frontend create-form field.

- Added Create Task Description support to Task Management and MVP Completion Status.

- Added PriorityLevel display/edit behavior to Priority and Sorting, Core Logic, and MVP Completion Status.

- Added DueDate edit behavior while preserving date-only MVP boundary.

- Added blank Progress Log save behavior to Progress Log and Core Logic.

- Added new use cases for required-field validation and validation feedback.

- Kept detailed automated test results, Manual MVP Workflow Testing details, and Practical UX Review checklist items in the Test Planning Document, not the SRS.

- Kept SRS limited to product requirements and MVP behavior.

- Removed references to separate project context documentation.

v1.10 (05/03/2026)

- Clarified that DueDate remains a date-only MVP field.

- Added time-specific due dates or due times to out-of-scope MVP items.

- Added date-only DueDate behavior to Due Date and Overdue Behavior.

- Added Date-Only DueDate Rule to Core Logic.

- Added time-specific due dates, due times, and date/time deadline behavior to Future Enhancements.

- Clarified that NextAction is editable through the existing task edit flow.

- Added NextAction creation/display/edit behavior to MVP Completion Status.

- Updated Routing implementation status to include /register route.

- Clarified that detailed test results, Practical UX Review checklist items, and Manual MVP Workflow Testing details belong in the Test Planning Document, not the SRS.

- Kept SRS limited to product requirements and MVP behavior.

v1.9 (05/01/2026)

- Added completion confirmation behavior before marking a task Completed

- Clarified that the confirmation supports the existing terminal completion rule

- Confirmed this does not add reactivation, restore, archive, undo, or any new MVP workflow

- Added frontend test coverage for cancelled and accepted completion confirmation behavior

- Verified Angular Karma test suite passed with 120 of 120 tests successful

- Manually verified completion confirmation behavior in browser

v1.8 (04/29/2026)

- Updated SRS version date after README and Test Planning Document alignment

- Kept SRS limited to product requirements and MVP behavior

- Clarified that completed tasks do not reactivate in the MVP

- Clarified create payload runtime behavior for backend-compatible priorityLevel mapping and blank dueDate handling

- Clarified status update endpoint as implemented backend architecture

- Preserved Practical UX Review and Manual MVP Workflow Testing as Test Planning Document activities, not SRS requirements

v1.7 (04/27/2026)

- Clarified that MVP feature development is complete

- Added current capstone work clarification without expanding MVP scope

- Added explicit out-of-scope items for new features, new workflows, advanced prioritization, UX redesign, and UI polish beyond basic clarity/access

- Clarified simplified single-user MVP approach

- Clarified NextAction frontend/API terminology

- Added frontend Task object shape and create payload distinction

- Added refreshTasks() refresh rule

- Added MVP Completion Status section

- Clarified that Practical UX Review and Manual MVP Workflow Testing belong in the Test Planning Document, not the SRS

v1.6 (04/25/2026)

- Marked frontend MVP as fully implemented
- Added full Progress Log behavior (UI + API + display)
- Corrected status transition rules to forward-only flow
- Updated Task List to reflect actual implemented behavior
- Clarified refresh-based UI update model
- Updated Use Cases to align with actual flows

v1.5 (04/23/2026)

- Implemented create-to-visible loop
- Introduced ngModel binding
- Removed DOM querying
- Updated architecture

v1.4 (04/22/2026)

- Updated frontend scope to include Login, Task List, and Task Create
- Updated authentication section to include login navigation behavior
- Updated architecture to reflect active Angular implementation
- Clarified frontend is integrated with backend APIs

v1.3 (04/20/2026)

- Added Integration and End-to-End Testing approach
- Added Playwright to testing environment
- Expanded scope to include full system testing (UI + API)

- Updated risks to include integration and browser testing concerns

v1.2 (04/18/2026)

- Added frontend Register UI to scope
- Updated architecture to include Angular implementation
- Clarified frontend-backend integration phase
- Updated risks to include Angular complexity considerations

v1.1 (04/16/2026)

- Implemented authentication endpoints
- Added Progress Log endpoints
- Added status update endpoint
- Clarified single-user MVP behavior